

PlatformAI

Riepilogo modifiche DevOps per il colloquio tecnico
Marco Bazzoli · Maggio 2026

■ Contesto

Il progetto **PlatformAI** (repository GitHub: *MarcoBaz/PlatformAI*) è un'applicazione reale composta da un backend **.NET 10** multi-progetto e un frontend **Angular**. È già deployata su Azure (App Service **IndustrialAI**, Resource Group **AuraAI**). Il deploy era gestito manualmente tramite uno script bash (*deploy.sh*).

In questa sessione è stata aggiunta l'intera infrastruttura DevOps professionale al progetto esistente, senza modificare il codice applicativo.

■ File aggiunti al repository

File	Cosa fa
Dockerfile	Build multi-stage: Node 22 (Angular) → .NET 10 SDK → runtime image
.dockerignore	Esclude bin/, obj/, node_modules, segreti e file CI dall'immagine
.azuredevops/pipelines/azure-pipelines.yml	Pipeline CI/CD YAML 6 stage per Azure DevOps
.azuredevops/pipelines/infra-pipeline.yml	Pipeline dedicata al deploy dell'infrastruttura Bicep
.azuredevops/pull_request_template.md	Template PR con checklist per i reviewer
infra/bicep/main.bicep	Template IaC: App Service, ACR, Key Vault, Managed Identity
infra/bicep/parameters/prod.bicepparam	Parametri produzione (AuraAI / IndustrialAI)
infra/bicep/parameters/staging.bicepparam	Parametri ambiente staging
docs/gitflow-strategy.md	Documento completo di branching strategy e branch policies

1 · Dockerfile — Build multi-stage (.NET 10 + Angular)

Il Dockerfile è composto da 4 stage distinti per ottimizzare la dimensione dell'immagine e la sicurezza:

- **Stage 1 — node-build:** installa le dipendenze npm con *npm ci* e compila Angular in modalità production. L'output finisce in *dist/fuse/browser/*
- **Stage 2 — dotnet-build:** copia prima solo i file *.csproj* (layer cache ottimizzato), esegue restore, poi copia il sorgente e compila l'intera solution in Release. L'output Angular viene copiato in *wwwroot/* prima del build.

- **Stage 3 — publish:** esegue `dotnet publish` per linux-x64, senza self-contained, producendo artefatti minimali in `/app/publish`
- **Stage 4 — final:** immagine ASP.NET runtime pura (senza SDK né Node). Utente non-root `appuser`, porta 8080, HEALTHCHECK su `/health`.

■ *Concetto chiave da spiegare al colloquio: il multi-stage build fa sì che nell'immagine finale non ci siano né il .NET SDK (~800MB) né Node. L'immagine runtime risultante è ~200MB.*

2 · Azure Pipelines YAML — 6 stage CI/CD

La pipeline `azure-pipelines.yml` si attiva su push a **main** e **release/***, e su PR verso **main** e **develop**. I segreti non sono mai nel YAML: vengono letti da Variable Groups collegati a Azure Key Vault.

Stage 1 — CI

- Installa Node 22 e .NET 10
- `npm ci + ng build --configuration=production` (Angular)
- Copia dist → wwwroot, poi `dotnet restore + build`
- `dotnet test` con code coverage (Cobertura/OpenCover) pubblicata su Azure DevOps
- SonarQube: Prepare → Analyze → Quality Gate (bloccante se fallisce)
- `dotnet publish + PublishBuildArtifacts` (artefatto 'drop')

Stage 2 — Security Scan

- Installa Trivy (scanner open source di Aqua Security)
- Scansiona il filesystem del repo per vulnerabilità HIGH e CRITICAL
- Produce un report SARIF pubblicato come artefatto
- Non blocca la pipeline (`continueOnError: true`), ma il risultato è visibile

Stage 3 — Docker

- `docker build + push` su Azure Container Registry (ACR)
- Tag doppio: `BuildId` (es. '142') + 'latest'
- Usa Service Connection ACR — nessuna password nel YAML
- Si esegue solo su branch `main/release`, non su PR

Stage 4 — Deploy Dev

- Deploy del container sul deployment slot 'dev' di IndustrialAI
- Smoke test automatico su `/health` dopo il deploy
- Collegato all'Environment 'platformai-dev' (tracciabilità deploy)

Stage 5 — Deploy Staging

- Deploy sul slot 'staging' (stesso App Service, URL diverso)
- Smoke test su `/health` — se fallisce blocca il passaggio a prod

Stage 6 — Deploy Prod

- Richiede approvazione manuale (gate configurato sull'Environment 'platformai-production')
- Deploy sul slot 'staging' dell'App Service di produzione
- Slot swap: `staging` → `production` (zero-downtime, rollback istantaneo)
- Smoke test finale su `https://industrialai.azurewebsites.net/health`

3 · Infrastructure as Code — Bicep

Il file `infra/bicep/main.bicep` descrive l'intera infrastruttura Azure come codice, parametrizzata per ambiente (prod/staging). Sostituisce la creazione manuale delle risorse dal portale.

Risorse provisionate:

- **Log Analytics Workspace + Application Insights** — telemetria e monitoraggio
- **Azure Container Registry (ACR)** — con `adminUserEnabled: false` (sicurezza)
- **User-Assigned Managed Identity** — identità condivisa tra App Service e le risorse Azure
- **Role Assignment: AcrPull** — la Managed Identity può fare pull da ACR senza password
- **Azure Key Vault** — con RBAC abilitato (no access policies), soft-delete attivo
- **Role Assignment: Key Vault Secrets User** — la Managed Identity legge i segreti da KV
- **App Service Plan** (Linux) + **App Service IndustrialAI** — configurato per container, httpsOnly, health check su /health
- **Deployment Slot 'staging'** — per lo slot swap zero-downtime verso produzione

Pattern chiave — Managed Identity (da spiegare al colloquio):

L'App Service usa una User-Assigned Managed Identity per autenticarsi ad ACR (pull immagini) e a Key Vault (lettura segreti). **Zero password** in appsettings, pipeline YAML o portale Azure. Nel codice .NET, `DefaultAzureCredential()` rileva automaticamente la Managed Identity in produzione.

4 · Gitflow & Branch Strategy

Il documento `docs/gitflow-strategy.md` definisce il modello di branching adottato nel progetto.

Branch model:

Branch	Scopo	Protetto?
main	Codice in produzione. Ogni commit = deploy.	■ Sì (2 reviewer)
develop	Integrazione feature. CI gira ad ogni push.	■ Sì (1 reviewer)
feature/*	Nuove funzionalità. Merge → develop via squash PR.	No
release/*	Stabilizzazione. Solo bug fix. Merge → main + tag.	No
hotfix/*	Fix urgenti da main. Back-merge su develop.	No

Branch policies configurate (per main):

- 2 reviewer obbligatori — almeno 1 CODEOWNERS
- Status check CI (`azure-pipelines.yml`) deve essere verde
- SonarQube Quality Gate deve passare
- Tutti i commenti PR devono essere risolti
- Work item Azure Boards collegato obbligatorio (AB#{id} nel commit)

Stato attuale del repo:

- Branch **main** esistente — collegato a origin/main su GitHub
- Branch **develop** creato localmente in questa sessione

■ Prossimo passo: *git push origin develop*, poi configurare *Branch Protection Rules* su *GitHub*

5 · Cosa sostituisce il `deploy.sh` manuale

Prima (<code>deploy.sh</code>)	Dopo (Azure Pipelines)
Build Angular manuale da terminale	Stage CI: ng build automatico ad ogni push
dotnet publish manuale	Stage CI: dotnet build + test + publish automatici
Nessun test eseguito nel deploy	Unit test con code coverage pubblicata su Azure DevOps
Nessuna analisi del codice	SonarQube Quality Gate integrato — blocca se la qualità scende
Nessun security check	Trivy scansiona vulnerabilità ad ogni build
az webapp deploy --type zip manuale	Docker image tracciata per build ID, deploy via AzureWebAppContainer
Deploy diretto in prod senza gate	3 ambienti (dev → staging → prod) con approval manuale
Nessun zero-downtime	Slot swap staging → prod: zero downtime, rollback in 30 secondi
Infrastruttura creata a mano dal portale	Bicep: infrastruttura riproducibile in qualsiasi subscription

6 · Passi successivi — Setup Azure DevOps

Per rendere operativa la pipeline sul progetto reale, sono necessari questi passaggi su Azure DevOps:

#	Azione	Dove
1	Creare il progetto su Azure DevOps	dev.azure.com → New project → PlatformAI
2	Collegare GitHub come Service Connection	Project Settings → Service connections → GitHub → MarcoBaz/PlatformAI
3	Collegare la subscription Azure	Project Settings → Service connections → Azure Resource Manager → 'Azure-ServiceConnection'
4	Creare o collegare un ACR	Portal Azure → Container Registry → platformaiacr (oppure Bicep lo crea)
5	Collegare ACR come Service Connection	Project Settings → Service connections → Docker Registry → 'ACR-ServiceConnection'
6	Configurare la SonarQube Service Connection	Project Settings → Service connections → SonarQube → 'SonarQube-ServiceConnection'

7	Creare Variable Group 'vg-platformai-common'	Pipelines → Library → + Variable group → aggiungere ACR_NAME, APP_SERVICE_NAME, RESOURCE_GROUP, SONAR_PROJECT_KEY
8	Creare gli Environments	Pipelines → Environments → platformai-dev, platformai-staging, platformai-production → su production aggiungere Approval check
9	Importare la pipeline	Pipelines → New pipeline → GitHub → PlatformAI → .azuredevops/pipelines/azure-pipelines.yml
10	Push branch develop su GitHub	Terminale: git push origin develop